

Chapter 8

Estimating User Stories

No project can go very long before someone starts asking “When will you be done?” The best approach for estimating stories would be one that:

- allows us to change our mind whenever we have new information about a story
- works for both epics and smaller stories
- doesn’t take a lot of time
- provides useful information about our progress and the work remaining
- is tolerant of imprecision in the estimates
- can be used to plan releases

Story Points

An approach that satisfies each of these goals is to estimate in *story points*. A nice feature of story points is that each team defines them as they see fit. One team may decide to define a story point as an ideal day of work (that is, a day without any interruptions whatsoever—no meetings, no email, no phone calls, and so on). Another team may define a story point as an ideal week of work. Yet another team may define a story point as a measure of the complexity of the story. Because of the wide variety of meanings for story points, Joshua Kerievsky has suggested that story points represent Nebulous Units of Time, or NUTs.¹

My preference is to treat a story point as an ideal day of work. We rarely have these ideal days, but thinking about stories in ideal time offers two advan-

1. Joshua Kerievsky on extremeprogramming@yahoogroups.com, August 5, 2003.

tages. First, it is easier than estimating in elapsed time. Estimating in elapsed time forces us to consider all other possible impacts on our time, such as the all-company meeting on Tuesday, my dentist appointment on Wednesday, a few hours a day for answering email, and so on. Second, estimating story points in ideal time gives our estimates a slightly better foundation than when they are estimated in entirely nebulous units. Since one of the main purposes of estimating is to be able to answer questions about the overall expected effort in a project, we will eventually need to convert estimates into time. Starting with ideal time makes that conversion a little simpler than starting with an entirely nebulous unit.

Estimate as a Team

Story estimates need to be owned collectively by the team. Later, in Chapter 10, “Planning an Iteration,” we’ll see that a story comprises multiple tasks and that a task estimate is owned by the individual who will perform the task. Story estimates, however, are owned by the team for two reasons: First, since the team doesn’t yet know who will work on the story, ownership of the story cannot be more precisely assigned than to the team collectively. Second, estimates derived by the team, rather than a single individual, are probably more useful.

Since story estimates are owned by the team it is important to have a reasonable portion of the team involved in coming up with the estimates. If the team is large (perhaps seven or more), not every developer needs to be involved, but generally the more developers involved the better. The customer participates while the programmers estimate, but she isn’t allowed to contribute her personal estimates or editorialize when she hears an estimate she disapproves of.

Estimating

My preferred estimation approach is derived from the Wideband Delphi approach documented by Boehm (1981). Just like Extreme Programming is an iterative approach to developing software, the approach we’ll use to estimating is an iterative approach to developing estimates. Here’s how it works:

First, gather together the customer and the developers who will participate in creating the estimates. Bring along the story cards and a stack of additional blank note cards. (Bring some blank cards even if you’re maintaining the story descriptions electronically.) Distribute a handful of blank cards to each partici-

pant. The customer selects a story at random from the collection and reads it to the developers. The developers ask as many questions as they need and the customer answers them to the best of her ability. If she doesn't know the answer, she takes a guess or asks the team to defer estimating that story.

▼

Everything Takes Four Hours

One of my favorite television shows was *Mad About You*, which is about a recently married couple living in New York. In one episode the husband (Paul Reiser) is being pestered by his wife (Helen Hunt) to go shopping for a couch. She insists the trip will take only an hour. He tells her that "Everything in the world takes four hours. You gotta go there, you gotta do whatever, eat, talk about where you should have eaten, and then come home. That's four hours minimum."

When the programmers estimate a story, they should include everything they'll need to do to complete the story. They need to factor in such things as testing their code, talking to the customer, perhaps helping the customer plan or automate acceptance tests, and so on. If they don't include these activities, it is like they are expecting shopping for a couch to take only an hour.

▲

When there are no more questions about the story, each developer writes an estimate on a card, not yet showing the estimate to the others. If the team has defined a story point as a day of ideal work, the developers think about how many ideal days the story will take to complete. If, instead, the team has defined a story point as, for example, the complexity of the story then the estimate is of the perceived complexity of the story.

When everyone has finished writing an estimate, the estimators turn over their cards or hold them up so everyone can see them. It is very likely at this point that the estimates will differ significantly. This is actually good news. If estimates differ, the high and low estimators explain their estimates. It's important that this does not come across as attacking those estimators. Rather, you want to learn what it is they were thinking about.

As an example, the high estimator may say, "Well, to test this story we're going to need to create a mock database object and that might take us a day. Also, I'm not sure if our standard compression algorithm will work and we may need to write one that is more memory efficient." The low estimator may respond with, "I was thinking we'd store that information in an XML file—that would be easier than a database. Also, I didn't think about needing more data—maybe that will be a problem."

At this point the group discusses it for up to a few minutes. Other estimators will undoubtedly have opinions on whatever reasons the high and low estimators were at the extremes. The customer clarifies issues as they come up. A note or two may be jotted on the story card. Perhaps a new story or two is written.

After the group has discussed the story, the developers again write their estimates on cards. When everyone has written a revised estimate, the cards are again displayed. In many cases the estimates will already converge by the second round. But, if they have not, repeat the process of having the high and low estimators explain the thinking behind their estimates. In many cases the high and low estimators will not be the same as in the first round. In fact, I have sometimes seen cases where the high and low estimators went to opposite extremes after gaining new knowledge during the discussion.

The goal is for the estimators to converge on a single estimate that can be used for the story. It rarely takes more than three rounds, but continue the process as long as estimates are moving closer together. It isn't necessary that everyone in the room turn over a card with exactly the same estimate written down. If I'm involved in an estimation meeting, and on the second round four estimators tell me 4, 4, 4, and 3 story points, I will ask the low estimator if she is OK with a four day estimate. The point is reasonableness not absolute precision. Yes, the developers could probably talk longer and reach consensus on three or four story points, but the time spent doing so isn't worth it.

Triangulate

After the first few estimates have been made, it becomes possible (and necessary) to *triangulate* the estimates. Triangulating an estimate refers to estimating a story based on its relationship to one or more other stories. Suppose a story is estimated at four story points. A second story is estimated at two story points. When the two stories are considered together the programmers should agree that the four-point story is roughly twice the size of the two-point story. Then, when they estimate a story as being three points, they should agree that it is roughly larger than the two-point story yet smaller than the four-point story.

None of this is exact, but triangulation is an effective means for a team to verify that they aren't gradually altering the meaning of a story point. A good way to triangulate is to pin story cards to the wall based on their size. Draw vertical lines on the wall, label each column with the number of story points, and then pin story cards to the wall in the appropriate column, as shown in Figure 8.1. As each new story is estimated, pin it in the appropriate location.

Very quickly compare the newly-estimated story to others in the column to see if it is “about the same.”

1	2	3	5	8	13
A user can...	A user can...	A user can...	A user can...	A user can...	A user can...
A user can...	A user can...	A user can...	A user can...		A user can...
A user can...		A user can...	A user can...		
A user can...			A user can...		

Figure 8.1 Pin story cards to the wall to facilitate triangulation.

Using Story Points

At the end of an iteration the team counts the number of story points they completed. They then use that as a forecast of how many story points they’ll complete in upcoming iterations of the same length. For example, suppose a team completes 32 story points over a two-week iteration. Our best guess is that they’ll also complete 32 story points in the next iteration. We use the term *velocity* to refer to the number of story points a team completes (or expects to complete) in an iteration.

Let’s look at how velocity and story points work together and why accuracy isn’t important. Suppose a team is starting a new project. They estimate all of the stories for the project and come up with a total of three hundred story points. Before the start of the first iteration they project that collectively they can complete thirty story points per week, which means the project will take ten iterations (weeks).

At the end of the first iteration the team adds up the story points for the stories they completed. They find that rather than completing 30, they completed 50 story points. If they can complete 50 story points in each iteration, they will finish the project in a total of six iterations. Should they plan on maintaining their measured velocity of 50? Yes, they should, based on three conditions.

First, nothing unusual (like a bunch of overtime, an extra programmer, or so on) affected productivity this iteration. The impact of overtime or other pro-

ductivity factors on velocity is obvious. If velocity of one iteration is based on everyone working sixty-hour weeks, there will be a big drop in velocity in the next iteration if they return to working forty hour weeks.

Second, the estimates need to have been generated in a consistent manner. This is important because it evens out fluctuations in velocity from one iteration to the next. Suppose in one iteration the team worked on nothing but stories that had been estimated only by a notorious overestimator. The team's velocity for that iteration would be artificially high if stories in later iterations were estimated by someone with a different bias. The best way to ensure that estimates are consistent is to use a team estimation process as described in this chapter.

Finally, the stories selected for the first iteration must be independent. This will be the case if stories were written in the manner suggested in Chapter 2, "Writing Stories." Consider an iteration comprised entirely of poorly constructed stories such that all work during the iteration is on the user interface. We would not want to extrapolate the velocity of that iteration onto all remaining iterations.



Why This Works

The Central Limit Theorem tells us that the sum of a number of independent samples from any distribution is approximately normally distributed.

For our purposes, this means that a team's story point estimates can be skewed way toward underestimation, way toward overestimation, or distributed in any other way. But when we grab an iteration's worth of stories from any of those distributions, the stories we grab will be normally distributed. This means that we can use the measured velocity of one iteration to predict the velocity of future iterations.

Naturally, the velocity of one iteration is not a perfect predictor. For example, an iteration containing one twenty-point story rather than twenty one-point stories would be a less accurate predictor. Similarly, velocity may change as a team learns new technology, a new domain, or becomes accustomed to new team members or new ways of working.



What If We Pair Program?

Whether or not a team chooses to pair program has no effect on story point estimates. For example, suppose a team with two developers is estimating in story points that are based on ideal days of work. They are not pair program-

ming. They plan a one-week iteration that will include two stories, each estimated at three story points. During the iteration they finish those two stories and calculate their team velocity as six.

Suppose instead that they pair program and that they estimate in ideal *pair* days. They look at the stories and decide that each will take two ideal pair days. At the end of the one-week iteration they have completed both stories and calculate their velocity as four.

While represented by different numbers, the velocities in these two cases are the same. The two teams are moving at the same speed since they completed the same amount of work in their iterations. This means that a team can choose to estimate story points in ideal pair days or ideal individual programmer days, and any differences will be reflected in the velocity.

▼

Precision Decreases as Story Size Increases

A problem with estimating in story points is that differences between some numbers can be hard to justify. For example, suppose the developers are considering a story and one developer suggests that the story is worth two story points. Another counters that the story is three story points. This discussion makes sense: three story points represents 50% more work than two story points. It's very possible for two developers to discuss a story and argue about differences of that magnitude.

However, now suppose the developers are arguing over whether a story should be seven or eight story points. In most cases a one point difference between numbers that large is too small to be discussed with any relevance. Arguing about whether a story is worth seven or eight story points implies more precision in our estimation process than we have.

To avoid this situation and simplify things, the team may want to agree to constrain estimates to specific pre-defined values such as:

$\frac{1}{2}$, 1, 2, 3, 5, 8, 13, 20, 40, 80

This is appealing because it reflects the truth that as estimates get larger, we know less about them. If the team has an epic to consider they'll have to decide whether it's a 40 or an 80, but they won't have to think about whether it's a 79 or an 80.

▲

Some Reminders

Working with story points can sometimes be confusing. Usually this results from thinking too hard about story points or trying to make them more than

they are. To reinforce the ways you can use story points, keep these facts in mind:

- Your team's story points are not equivalent to my team's story points. A story your team estimates as worth three story points may be worth five to my team.
- When a story (possibly an epic) is disaggregated into its constituent stories, the sum of the estimates for the individual stories does not need to equal the estimate of the initial story or epic.
- Similarly, a story may be disaggregated into constituent tasks. The sum of the estimates for the tasks does not need to equal the estimate of the initial story.

Summary

- Estimate stories in story points, which are relative estimates of the complexity, effort or duration of a story.
- Estimating stories needs to be done by the team, and the estimates are owned by the team rather than individuals.
- Triangulate an estimate by comparing it to other estimates.
- Whether or not a team programs in pairs has no impact on story point estimates. Pair programming affects the team's velocity, not their estimates.

Developer Responsibilities

- You are responsible for defining story points in a manner that is relevant and usable by your team. You are responsible for consistently sticking to that definition.
- You are responsible for giving honest estimates. You are responsible for not giving in to temptation or pressure to give low estimates.
- You are responsible for estimating as a team.
- You are responsible for giving estimates that are consistent with other estimates. That is, all the two-point stories should be similar.

Customer Responsibilities

- You are responsible for participating in estimation meetings, but your role is to answer questions and clarify stories. You are not allowed to estimate stories yourself.

Questions

- 8.1 During an estimating meeting three programmers are estimating a story. Individually they estimate the story at two, four and five story points. Which estimate should they use?
- 8.2 What is the purpose of triangulating estimates?
- 8.3 Define velocity.
- 8.4 Team A finished 43 story points in their last two-week iteration. Team B is working on a separate project and has twice as many developers. They also completed 43 story points in their last two-week iteration. How can that be?

This page intentionally left blank